

리스트 (List)

1. 리스트의 개념

- 리스트(또는 선형 리스트)는 **순서를 가진 항목들의 모임**.
- 일상에서의 리스트 예: 요일, 한글 자음, 카드, 할 일 목록 등.

2. 리스트의 연산

- 항목 추가 (삽입)
 - 끝, 처음, 중간에 추가
- 항목 삭제
 - 특정 위치 또는 전체 삭제
- 항목 대치
- 항목 탐색
 - 특정 항목 포함 여부 확인
 - 특정 위치의 항목 조회
- 리스트 상태 확인
 - 비어있는지(empty), 꽉 찼는지(full)
- 항목 출력
- 항목 개수 카운트

3. 리스트 ADT 함수 설계

- `insert(list, position, item)`
- `delete(list, position)`
- `getEntry(list, position)`
- `isEmpty(list)`
- `isFull(list)`
- `printList(list)`
- `initList(list)`

4. 리스트 구현 방법

4.1 배열(Array)을 이용한 구현

- **장점:** 구현 간단
- **단점:**
 - 항목 개수 제한
 - 삽입/삭제 시 이동 필요 → 오버헤드 발생
- 배열은 **순차적 표현**

4.2 연결 리스트(Linked List)를 이용한 구현

- 노드는 **데이터 필드 + 링크 필드**로 구성
- **장점:**
 - 동적 메모리 할당으로 크기 제한 없음

- 삽입/삭제 연산 효율적 (이동 불필요)
- 단점:
 - 포인터 사용으로 구현 복잡, 오류 발생 가능성 높음
- 표현 방식:
 - 단순 연결 리스트
 - 원형 연결 리스트
 - 이중 연결 리스트

5. 연결 리스트 상세

- 노드 구조:

```
typedef struct ListNode {  
    element data;  
    struct ListNode* link;  
} ListNode;
```

- 자기참조 구조체 사용
- **헤더 포인터**: 첫 번째 노드를 가리키는 포인터
- 동적 메모리 할당 (**malloc**) 사용

6. 주요 개념 요약

- 배열 기반 리스트는 메모리에 연속적으로 저장됨
- 연결 리스트는 메모리 상에서 **분산 저장**
- 삽입/삭제에 따른 포인터 조작이 핵심
- 노드 연결 방식에 따라 다양한 연결 리스트 존재
- **중간고사 대비**: 교재 183~189쪽 정독 필요